Laboratório VISGRAF
Instituto de Matemática Pura e Aplicada

The N-Surface Method:
Rigor and Spontaneity in Human–AI Research

Luiz Velho, Claude, GPT

Technical Report TR-26-12 Relatório Técnico

October - 2026 - Outubro

The contents of this report are the sole responsibility of the authors.
O conteúdo do presente relatório é de única responsabilidade dos autores.

The N-Surface Method: Rigor and Spontaneity in Human–AI Research:

Luiz Velho
with Claude (Anthropic) and GPT (OpenAI)
VISGRAF Lab, IMPA

Technical Note · draft 8 October 2026 · revised 12 October 2026

Abstract

Between August and October 2026, one research line on active foveal 3D vision at VISGRAF/IMPA ran through six successive repositories. Each had its own protocol for collaboration between one human researcher and AI systems. The configuration grew from a single conversational assistant to combinations of surfaces from two vendors (ChatGPT, Claude Chat, Claude Code), spread over two machines and several interfaces, with GitHub as the shared record.

This note reconstructs that evolution and extracts a method from it. The protocols changed a great deal; a small kernel did not. Every number is labelled proposed, measured or assumed. Every tool ships a check that can fail. The repository, not the conversation, is the memory. History moves forward only. The human decides.

Around the kernel sits an adaptive shell: the number and roles of surfaces, the transport between them, commit authority, ceremony per kind of step, cost classes and loop time scales. Its settings were re-derived each time platforms, capabilities or the project stage changed. We describe surfaces by capability rather than by model, show where the balance between rigor and spontaneity was struck wrongly in each direction, and list the method's practical limitations.

Section 9 reports a first trial of three proposals for seamless adaptation, in f3d-vision on 12 October 2026: the protocol split into a kernel, a shell of dial settings and the project state, a coherence lint, and committed records of the process.

We call the result an **N-Surface method**. N is not a target; it is whatever the current context requires.

1 The problem

AI collaboration makes implementation, experiments and prose cheap; verification and scientific judgment stay scarce. In software the characteristic failure is a bug. In research it is a wrong belief: a plausible claim, backed by a computation that may be correct, read as evidence for something it did not establish. AI collaboration can produce internally consistent artifacts faster than one person can check what they warrant.

A second problem appears once more than one AI system is involved: **the collaboration itself moves**. Surfaces differ in what they can see, change and run. Platforms change those capabilities without notice. A GitHub connector that wrote yesterday is read-only today. The project also passes through stages (exploration, science, refactoring, presentation) that need different amounts of discipline. A fixed protocol over-constrains some stages and under-constrains others.

The method therefore has to strike two balances:

- **Rigor versus spontaneity.** How much ceremony a given step deserves, so that errors surface without suppressing cheap exploration.
- **Stability versus adaptivity.** Which parts of the protocol may change with the context, and which must not.

Terms

- **Surface.** An AI system reached through a specific interface, with specific access. Claude Chat in a browser and Claude Code in VS Code on a GPU workstation are different surfaces, even when the underlying model is similar.
- **Station.** A role in the working loop: deciding, specifying, executing, reviewing, recording. A surface or the human may occupy one or several. Station counts below follow each protocol's own naming.
- **Protocol carrier.** The file that states the rules for a repository: `README.md`, `CLAUDE.md`, `AGENTS.md`.

Scope

One researcher, one research line, about two months, six repositories. The sample size is one. This is not a validated general method but an account specific enough to be checked. All six repositories are public (Appendix B).

2 The journey

Six repositories in about two months, each a restart with a revised protocol. The protocols that named their stations counted four (bio-3d-vision), then five (fov-3d-vision), then three (f3d-vision): the method matured by compressing, not by accumulating.

Stage 1 · bioeye: one entity

ChatGPT alone, through conversation and git, built a thin vertical slice of a six-layer active Bayesian stereo framework: a synthetic scene, two matchers, a Blender scene generator and a greedy gaze policy. The only protocol was the README, which is a data contract (files, units, run commands). It reached a running demo and slides quickly. It closed the accumulation loop but not the perception–action loop, and knowledge lived in the conversation. The human was author, executor and courier.

Stage	Repository	Start	Surfaces	Protocol carrier	Human role	Default stance
1	bioeye	9 Aug	ChatGPT	README (a data contract)	Author, executor	Implicit
2	active-stereo	15 Aug	Claude Chat + Code	CLAUDE.md "constitution"	Approves every commit	Prevention
3	bio-3d-vision	29 Aug	Claude Chat + Code (+ Cowork, unused)	CLAUDE.md, workflow, state file, ledgers	Gate and relay	Epistemic rigor
4	w3d-scenes	12 Sep	Claude Chat, workstation check	README	Executor of checks	Controls first
5	fov-3d-vision	12 Sep	Claude Chat + Code; ChatGPT at phase ends	CLAUDE.md working agreement	Courier (zip + sha256)	Cheap recovery
6	f3d-vision	25 Sep	ChatGPT + Claude Code; from 11 Oct Claude Chat + Code, ChatGPT at phase ends	CLAUDE.md + AGENTS.md; from 12 Oct kernel, shell and handoff	Decides; carries one prompt	Science-led, engineering-disciplined

Table 1: Six stages, six repositories.

Stage 2 · active-stereo: the constitution

Thinking and execution were split between Claude Chat and Claude Code. `CLAUDE.md` became a normative constitution: layers as module boundaries, three structural closures, invariants on units, frames, invalid data, determinism and uncertainty, and a list of prohibitions. Code never committed without being asked. Two ideas from this stage lasted:

- **The handoff contract.** A number arriving from Chat is a hypothesis with a pointer; the receiver holds the files and verifies. The reverse direction is deliberately asymmetric: Code discloses which inputs were measured or assumed; Chat may not use a Code number to settle a dispute without asking what it was measured against.
- **Memory is a cache, never the record.** A lesson kept only in an assistant's memory does not exist for the project.

Stage 3 · bio-3d-vision: the research method

A four-station loop: the human sets the goal and gates, Chat specifies, the human relays, Code executes, and a separate Chat checkout verifies. The rules reacted to stage 2. **Carry only what can fail.** Decision records are written after the thing is tried. Units and frames live in docstrings, beside the code they describe. Inherited numbers carry provenance. A `spikes/` lane allows rough tries without ceremony. A task is done only if it moved something.

Stage 4 · w3d-scenes: a convenience step

Part convenience, part data migration: gathering test scenes (an HDRI, a generated calibration room, the Classroom demo) with 360° previews and checks. Claude Chat built it in a CPU sandbox and the workstation verified it on the GPU; it has three commits and was absorbed the same day as step A1 of fov-3d-vision. Its README already separates *verified on CPU*, *verified on GPU* and *not yet tested*, and ships positive and negative controls. Small as it is, it prefigures the later claim states.

Stage 5 · fov-3d-vision: engineering first

The stance reversed: "A visual result at every step. Mistakes are expected," so the process optimises for cheap recovery, not prevention. Chat wrote code and docs, ran what its sandbox allowed (self-tests, negatives, a stub Blender) and handed over a zip with its sha256 and a prompt for Code. The prompt listed commands with cost classes, expected checks, likely failures, what must not be changed and what to report. Luiz applied and committed; Code ran, diagnosed, pushed to `main` and reported. No PR gate; revert if wrong.

New elements: two hard rules (measured or assumed; a check that can fail), cost classes with small and full profiles, references treated as pinned assets, and two Python interpreters kept apart. A second, undocumented loop ran at phase boundaries: Claude Chat summarised the phase, ChatGPT reviewed it, and the review went back to Chat (Section 5).

Stage 6 · f3d-vision: GitHub carries state

ChatGPT on a Mac Studio, Claude Code in VS Code on an Ubuntu RTX 4090 workstation, GitHub between them. The working agreement went through four versions in under three weeks:

1. Chat commits proposals to the active branch when explicitly authorized.
2. After an OpenAI restriction, ChatGPT can no longer write to the repository.
3. ChatGPT is read-only; it designs, reviews and prepares a paste-ready prompt. Claude Code creates the branch and an isolated worktree, commits the contract before executing, implements, measures and pushes. Luiz carries one prompt and decides.
4. From 11 October: Claude Chat replaces ChatGPT as the design and review surface, and may push contracts, documentation and host-side code to the work branch. Claude Code keeps execution and measurement. On 12 October ChatGPT returned as an optional reviewer at phase ends, which restores the configuration of fov-3d-vision. The same day the agreement was restructured into three layers (Section 9).

New elements: three claim states (proposed, measured, assumed); "migrate behavior first; redesign structure second"; a Chat Handoff file at accepted milestones; separate `previews/` and `visuals/` trees. The tagline moved from "engineering-first / science-second" to "science-led, engineering-disciplined". The stage produced the research sequence summarised in Section 6, an official baseline reproduced bitwise, and an accepted technical report, *Active Foveal 3-D Vision Playground* (TR-11-2026) [2].

3 Surfaces as capability profiles

A surface is defined by what it can see, change, run and remember; which model sits behind it matters less. Roles in the protocol follow from these capabilities, and when a capability changes, the roles have to be re-derived.

Participant	Sees	Changes	Runs	Remembers	Role in stage 6
Luiz (human)	Everything, at limited bandwidth	Anything	Anything	Across all repos and stages	Sets direction, decides, routes
ChatGPT (Mac, app or browser)	The live repo, read-only	Nothing in the repo	A sandbox: no GPU, no real assets	The conversation	Until 10 Oct, designs, specifies, reviews, writes the Code prompt; from 12 Oct, an optional reviewer at phase ends
Claude Chat (browser or app)	Files and excerpts passed to it; public repos; from 11 Oct, a clone of the live repo	Nothing in the repo until 10 Oct; then the work branches Luiz names	A CPU sandbox, sometimes a stub Blender	The conversation; a claude.ai project memory	Specified and reviewed in stages 2–5; from 11 Oct, plans, specifies and reviews steps again
Claude Code (VS Code, Ubuntu RTX 4090)	The working tree	Branches, worktrees, commits	GPU, Blender, full assets	What the repo holds; on the workstation, an auto-memory (Section 9)	Executes, measures, mutates the repository
GitHub repository	—	—	—	Code, contracts, reports, history	The durable record and the handoff medium

Table 2: Surfaces described by capability.

Four principles follow from the table.

Authority follows access. The surface that holds the files and the hardware overrules the surface that holds the memory. Only execution on the workstation turns a proposed number into a measured one. GPU, Blender and performance claims made anywhere else stay proposed.

Same model, different surface, different station. Claude Chat and Claude Code differ in access, not intelligence, and that difference decides who may claim what. Surfaces from different vendors differ in their priors as well, which makes them useful reviewers of each other.

The human is a router, and transport sets the routing load. Copy-paste, a relayed specification, a zip with its checksum and a git commit cost the human very different amounts of work. Each change of transport changed how much of the human's time went into carrying files instead of deciding.

Platform policy is part of the topology. When ChatGPT lost write access, the protocol consolidated all repository mutation in Claude Code and reduced the human's carrying to one prompt. Making the repositories public was the same kind of decision: it let surfaces from both vendors read the same state despite access restrictions.

4 The invariant kernel

Five rules survived every protocol change from stage 2 onward, reworded but not weakened; a sixth arrived late and has held since. Together they make a claim checkable regardless of which surfaces produced it. Each rule is listed with the failure that motivates it, because a rule without its reason is easily simplified away as bureaucracy.

Rule	First form	Final form	Failure it prevents
Every claim is proposed, measured or assumed	Chat numbers are hypotheses with pointers (stage 2)	Three named states; only workstation execution makes a number measured	A fluent figure from a summary becomes a fact
Every tool ships a check that can fail	"Carry only what can fail" (stage 3)	A control, known answer, invariant or behavioral comparator per tool	A green test that could never have come out red
The repository is the memory	"Memory is a cache, never the record" (stage 2)	Contracts, reports and a Chat Handoff index in the repo	Knowledge lost when a conversation ends
History moves forward only	No force-push; append-only decisions (stage 2)	Commit, push, revert if wrong	A correction that erases how the error escaped
The human decides	Ask when the spec is ambiguous (stage 2)	Explicit authority; stop conditions return decisions to Luiz	An AI resolving ambiguity by inference
The contract precedes execution	Pre-registered experiments (stage 3)	The contract is committed before implementing: by Code until 10 Oct, by Claude Chat since	Tuning against the result; prevented errors leaving no trace

Table 3: The invariant kernel.

The kernel is deliberately small. It says nothing about which surfaces exist, how files travel, or how much ceremony a step needs. Those belong to the shell (Section 5).

The kernel is also what makes spontaneity safe. "Mistakes are expected" works only when mistakes become visible; claim states and fail-capable checks are what make them visible. Without the kernel, cheap recovery is just speed.

5 The adaptive shell: dials and triggers

Everything outside the kernel was adjusted at least once. Treating these as dials, rather than as fixed rules, is what makes the method adaptive: changing one setting should not require rewriting the constitution.

Dial	Settings observed, in order	What moved it
Surfaces and roles	One assistant → Chat + Code → Chat specify / Code execute / Chat verify → plus a cross-vendor phase reviewer → ChatGPT design + Code execution → Claude Chat design + Code execution, ChatGPT optional phase reviewer	Need to separate thinking from running; need for an independent reviewer; platform access
Transport	Copy-paste → relayed, uncommitted specs → zip + sha256 → git commits by both AIs → git, read-only Chat, one paste-ready prompt → git, Claude Chat pushing to work branches	Courier load on the human; write restrictions
Commit authority	Human only → never without asking → Code pushes to `main` → Code on the named branch, in an isolated worktree → Claude Chat on work branches too; main moved by either, on Luiz's acceptance	Speed versus safety of the shared checkout
Ceremony per step	Uniformly heavy → uniformly light → by step type	Exploration suppressed (stage 3); refactoring needing proof of preserved behavior (stage 6)
Evidence per step type	Scientific: measurable result, visual, checks, report · Design: reviewed specification · Refactoring: comparator with `MISMATCHES 0`	Project stage
Cost classes	Absent → interactive (<10 s), batch (<5 min), overnight (justified in advance); small and full profiles	Expensive renders stalling the loop
Where conventions live	Constitution → docstrings → constitution again → kernel and shell (Section 9)	A false clause in the constitution (stage 2); cross-vendor surfaces needing one readable place (stages 5–6)
Loop time scales	One loop → steps plus phases → steps reviewed per commit → steps plus optional phases again	Need for review at a different distance from the work
Visual record	Demo figures → each iteration's artifact beside the last → a visual every step → `previews/` (machine, per stage) and `visuals/` (human, persistent)	Need to compare by looking; disk and lifetime of evidence

Table 4: The dials of the adaptive shell.

Triggers

Five kinds of event moved the dials:

1. **A platform capability changed**: write access removed, a connector added, repositories made public.
2. **A failure was observed**: a false constitution clause, an unused spikes lane, an uncommitted specification.
3. **The project changed stage**: exploration, science, migration and refactoring, presentation.
4. **Execution cost changed**: a 35-minute reference render cannot sit inside an interactive loop.
5. **The human's bandwidth ran out**: carrying zips between machines is time not spent deciding.

Two time scales

The most consequential shell setting was never written down. In fov-3d-vision two loops ran on different time scales. The inner **steps** loop went from Chat's plan through the human to Code's execution and back through the repository. The outer **phases** loop took Chat's end-of-phase summary to ChatGPT for review and suggestions, and returned the outcome to Chat for reassessment.

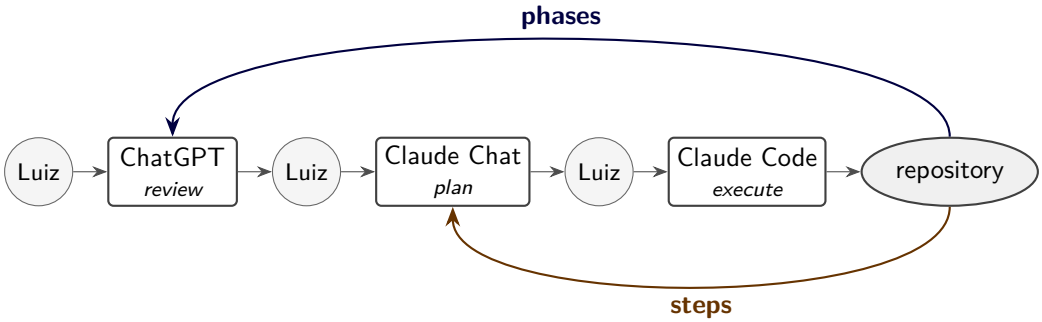


Figure 1: fov-3d-vision working loops, redrawn from the hand sketch. The inner steps loop returns to Claude Chat through the repository; the outer phases loop returns to ChatGPT for review.

Chat is Claude Chat and Code is Claude Code. The phases loop with ChatGPT ran in practice but was never written into the protocol.

The outer loop placed a reviewer from another vendor at stage boundaries, which is where unexamined momentum is most expensive. A reviewer that did not write the plan does not share the planner's momentum. In f3d-vision the reviewer moved inward: ChatGPT reviews every committed result, so phase-level and step-level review collapsed into one loop. From 12 October the two loops are back: Claude Chat reviews steps, ChatGPT optionally reviews phases, and both are now written into the protocol (Section 9).

6 Rigor versus spontaneity in practice

Neither uniform rigor nor uniform informality worked. The balance came from specific, bounded mechanisms that let the kernel stay strict while individual steps stayed cheap. Figures below are as recorded in the f3d-vision repository.

When rigor went too far

In bio-3d-vision every question became a pre-registered experiment with falsifiers, provenance, findings and a verdict. The low-ceremony `spikes/` lane existed from day one, was enforced, and was used zero times in thirteen experiments. After enough formal work, a rough script no longer felt like a sanctioned move; it felt like a lapse. The record also favoured ceremony, because formal work leaves artifacts and a successful spike is meant to be deleted.

When spontaneity went too far

In bioeye, speed came without a loop being closed or a record being kept. In stage 5, pushing to `main` and reverting if wrong worked, but only because the two hard rules made each mistake visible. Spontaneity is safe above the kernel and unsafe without it.

Mechanisms that combine the two

- **The bounded bet.** After two negative correspondence experiments, natural stereo got exactly one more attempt: one frozen SGBM configuration, no tuning, both outcomes mapped to actions in advance. It was not adopted, natural stereo was deferred, and the project returned to its main active-perception line. The framing was strict; the attempt was cheap.
- **Negative results count.** AB1a, AB1d and AB1d3 were accepted as negative experiments, each with a stated mechanism. Accepting a negative removes the pressure to tune until something passes.
- **A cheap control before a redesign.** Rather than redesign the matcher after AB1d, AB1d2 changed only observation quality (256 $\rightarrow$ 4096 samples per pixel). Correct correspondences within one pixel rose from 9.2 / 16.8 / 9.2% to 35.9 / 66.6 / 49.2%. One variable, one run, one clear answer.
- **A simple baseline first.** Greedy Foveal Explorer v0 is a deliberately simple loop: fixate, reconstruct locally, update one global map, saccade. It stopped at fixation 542 with 99.00% of the sphere seen and 98.28% with depth, while the elaborate controller line (NS1e) remained under review. It became the official baseline.
- **Mandate errors are named.** NS1c faithfully implemented its contract, but the contract placed a gate in the wrong phase. It was reviewed, not accepted and not merged; NS1c2 changed that one variable and was accepted. Separating an error of design from an error of implementation keeps faithful work from being blamed and wrong work from being accepted.

- **Cost classes.** Development happens on the small profile, in seconds; the full profile runs once, for the number that gets reported. Spontaneity is cheap where iterations are cheap.

The working principle

Scale ceremony to the cost of being wrong and to how hard the error would be to undo. Where recovery is cheap and the kernel makes errors visible, try first. Where a result will become a premise for later work, declare the criteria first.

7 Practical limitations of the method

Six limitations surfaced in practice. Some were accepted knowingly in the heat of the work, on a cost–benefit judgment; others are imposed by vendor policy and technology and must simply be lived with. Each is a lesson for the next version of the method. A common thread runs through them: the method records artifacts well and the process that produced them less well. Limitations 1–3 are the target of the first trial reported in Section 9.

1. **The process that worked best left no record.** The phase-level review in fov-3d-vision began because Luiz felt the need for a second opinion at a stage boundary, and it worked: a discovery emerged from the cross-vendor review. It was not written into the protocol.

2. **Governance files lack a coherence check.** Two kinds of check are possible, and they differ as semantics differs from syntax and grammar. Judging whether a protocol's content serves its purpose is a meta-review: hard to automate and perhaps not needed, since the human sets that purpose. Checking clarity and coherence is closer to a lint, and it is wanted.

3. **The constitution absorbs state.** Most of the final `CLAUDE.md` is a running scientific record: experiment outcomes, numbers, branch states. Chat began adding project state there at some point; it was harmless at the time and was left in place. The Chat Handoff came later, and the two were never reconciled. The cost is a long file, partly duplicating the handoff. Fixing this remains a target for the next version.

4. **Vendor constraints load the human courier.** Every topology aimed to minimise human file shuffling. But vendor policy and technology (write access, cross-platform access) are outside the project's control. The final protocol limits it to one prompt and turns that prompt into a committed contract; the prompt's fidelity to the contract is checked only by the human in the loop, Luiz.

5. **Continuity lives in the human, and mostly should.** Six repositories in two months gave each stage a clean start; continuity across them was carried by Luiz. High-level continuity (the question, the direction, what was learned) is natural for the human to hold. Mid- and low-level continuity (numbers, criteria, conventions) can be enforced explicitly where the human judges it necessary.

6. **Moving between formality and informality needs trust, not only rules.** Formal and informal work each need a proper setting, but above all a working trust and companionship between human and machine, with its inherent asymmetry. One side decides and bears responsibility; the other proposes and executes under delegated authority. Rules can make that trust safe; they cannot create it. This is perhaps the central philosophical question the method raises, and it is left open here.

8 Toward seamless adaptation

So far every adaptation happened by rewriting the protocol in a new repository or a new version of `CLAUDE.md`. Seamless adaptation would mean changing a dial without touching the kernel, and leaving a record of why. Four proposals follow. The first three were tried once, in f3d-vision (Section 9); the fourth has not been tested.

1. **Three layers, three files.** Keep the kernel in one short file that rarely changes. Keep the shell's current dial settings in a second file, each with the trigger that set it. Keep project state in the Chat Handoff, never in the constitution. A capability change then edits one line in the shell file.
2. **A coherence lint for the protocol.** A small lint over the governance files for clarity and coherence, not content: statements about write access agree with the transport described, contract paths match the repository layout, the named branch exists, terms are used consistently. Whether the protocol serves its purpose stays a human judgment. This applies the kernel's second rule to the syntax of the method, not its semantics.
3. **Commit the process, not only its products.** Record phase reviews, rejected contracts and prevented errors as small committed files. The cheapest saves are currently the least visible.
4. **State the kernel once for every agent.** `AGENTS.md` already points all coding agents to one canonical file. Testing a further surface (Codex is planned) will show whether the kernel is truly vendor-neutral or quietly assumes Claude Code.

Open questions

- Do the observed failures belong to the method or to this researcher? Only a second person running it can tell.
- What is the smallest N that serves a given stage? Exploration may need one surface; validated science, at least an executor and an independent reviewer.
- Can cross-vendor review replace some human review at phase boundaries, or does it only move momentum from one model to another?
- How should a project record work it correctly prevented?
- Does the method transfer to research that is qualitative or construction-driven, where falsifiers are harder to state?

9 A first trial: Governance-1

Proposals 1–3 were tried once, in f3d-vision on 12 October 2026, as one governance step. Claude Chat wrote the contract and the implementation, Claude Code executed it on the workstation, and Luiz accepted it; the commits span 16:13 to 16:51. The configuration was the one proven in fov-3d-vision: `[ChatGPT >] Luiz > ClaudeChat > Luiz > Claude Code`. Figures are measured, from the step's committed report; line counts are at commit `11bfbf6`.

What was built

Layer	File	Holds	Size
Kernel	`CLAUDE.md`	authority, the six kernel rules, contracts, reports, repairs, project invariants	1,023 → 111 lines
Shell	`docs/process/shell.md`	surfaces and their write access, who commits what, 16 dials each with its trigger	104 lines, new
State	`docs/chat-handoff.md`	where the project is now, rewritten at each milestone	2,557 → 98 lines
Lint	`tools/process/lint.py`	checks of form, not content, with a self-test of 22 mutations	273 lines, new
Records	`docs/process/records/`	one small file per review, rejected contract or prevented error	4 records, new

Table 5: What Governance-1 built.

The kernel imports the shell and the handoff, so Claude Code loads all three at startup; other surfaces read them in that order. Reading at session start fell from about 3,600 lines to about 310. The replaced files survive byte-identical in a history folder.

What it cost

One shell file, one lint script, one folder. In the loop: a one-second lint before any push that changes a governance file, a small file when something is caught, and a handoff rewritten instead of appended. The two dials Luiz added at acceptance, a fast-forward of the shared checkout after each acceptance and assistant memories kept as caches, never status, edited the shell and left the kernel as it was. That is the seamless adaptation of Section 8, on its first two occasions.

Limits of the trial

One repository and one afternoon, judged by the surfaces that built it. The lint was written knowing the defects it would find. The cross-vendor phase review is configured but has not run yet. The 150-line caps are a choice, not a finding; Claude Code's documentation advises under 200 lines per `CLAUDE.md`. Whether records keep being written once the novelty passes is the real test, and that takes months.

References

[1] Luiz Velho and Claude. "Experiments on Active 3D Vision". Technical Report TR-09-2026, VISGRAF Lab – IMPA, 2026.

[2] Luiz Velho and Claude. "Active Foveal 3-D Vision Playground". Technical Report TR-11-2026, VISGRAF Lab – IMPA, 2026.

A Seven epistemic failure modes

These were identified in bio-3d-vision (stage 3) and shaped the kernel. Each is a way to acquire a wrong belief from correct-looking work.

Failure	What goes wrong	Example from stage 3	Defense
Fluent wrong claims	A plausible statement survives reading and fails against its source	Of 18 substantive claims in the report, 3 had wrong figures and 8 were true only under omitted conditions	Verify at the source; never ratify a figure from a summary
Stale state	A premise true when a specification was written is false when it runs	Specifications written against one repository state, executed against a later one	Name the assumed commit in every contract; reject on mismatch
Work that cannot fail	A criterion is satisfied by construction	A threshold based on seed-to-seed spread could test whether an effect exceeded scene variation, not whether it existed; ten verdicts were read through the stronger meaning	Pre-register a falsifier for the specification itself, allowing persist, shrink, grow or reverse
Inherited assumptions	Carried methods acquire the authority of local results	The faulty criterion crossed from a predecessor repository unmarked, while inherited numbers were marked	Mark carried methods as inherited at first use
Conversation-only knowledge	Load-bearing state exists only while a session lives	A detailed plan shaped several tasks without being written down	The repository is the memory; run cold-session tests
The authored instrument	A synthetic fixture fails exactly where the question lives	A depth-warped stereo stimulus produced false matches at depth discontinuities that passed the validity test	Characterise a stimulus as an instrument before measuring with it
Momentum	Clean progress stops the framing from being questioned	The faulty criterion survived a long run of clean results	An occasional devil's-advocate pass at stage boundaries

Table 6: Seven ways to acquire a wrong belief from correct-looking work.

The last defense was proposed in stage 3. In stage 5 it was realised in a different form: a reviewer from another vendor at phase boundaries (Section 5).

B Repositories

All six are public. Each claim in this note about a stage can be checked against its repository.

Repository	Start	Surfaces	Scientific scope	Output
visgraf/bioeye	9 Aug 2026	ChatGPT	Euclidean framework, layers 1–5	Demo and slides
visgraf/active-stereo	15 Aug 2026	Claude Chat + Code	Spherical, layers 1–3, Bayesian	11 experiments
visgraf/bio-3d-vision	29 Aug 2026	Claude Chat + Code	Spherical, layers 1–5, closing the loop	13 experiments; TR-09-2026 [1]
visgraf/w3d-scenes	12 Sep 2026	Claude Chat; workstation verification	Test scenes	Archived; superseded by step A1 of fov-3d-vision
visgraf/fov-3d-vision	12 Sep 2026	Claude Chat + Code; ChatGPT phase review	Foveated omnidirectional rendering and 3D reconstruction	Phase reports
visgraf/f3d-vision	25 Sep 2026	ChatGPT + Claude Code; from 11 Oct Claude Chat + Code, ChatGPT phase review	Foveated omnidirectional segmentation and reconstruction	Greedy Foveal Playground baseline; TR-11-2026 [2]; Governance-1 (Section 9)

Table 7: The six repositories.